

Final Capstone Project Report

Integrated AI Real Estate Advisory System

Team Members

Shourya Bafna

Aditya Bharadwaj

Daksh Batra

Om Yadav

Project Milestones

Milestone 1: Predictive ML & Data Engineering

Milestone 2: Agentic AI & RAG Orchestration

Abstract

This report explains how we built a real estate advisor that combines math-based price predictions with an AI chat system. By using historical data to predict house prices and an AI agent to research the market, we created a single application that can give both accurate price estimates and helpful investment advice.

April 19, 2026

Contents

1	1. Introduction & Problem Statement	3
1.1	1.1 The Real Estate Data Fragmentation Problem	3
1.2	1.2 Objective & Project Overview	3
2	2. Milestone 1: Data Engineering & EDA	4
2.1	2.1 The Ames Housing Dataset	4
2.2	2.2 Missing Data Handling & Categorical Encoding	4
2.3	2.3 Exploratory Data Analysis (EDA)	4
3	3. Milestone 1: ML Predictive Modeling	5
3.1	3.1 Algorithm Selection: Random Forest Regression	5
3.2	3.2 Model Training & Tuning	5
3.3	3.4 Evaluation Visualizations	5
4	4. Milestone 2: Transitioning to Generative AI	6
4.1	4.1 The Need for Agentic Architectures	6
4.2	4.2 Framework Selection: LangChain & LangGraph	6
5	5. Milestone 2: Multi-Node Orchestration	7
5.1	5.1 AgentState Dictionary	7
5.2	5.2 System Architecture	7
5.3	5.3 The 5 Nodes	7
6	6. Milestone 2: Knowledge Base & RAG	9
6.1	6.1 Grounding the AI	9
6.2	6.2 ChromaDB Integration	9
6.3	6.3 RAG Mathematical Routing	9
7	7. Application UI & Experience Design	10

7.1	7.1 Component Modularity	10
7.2	7.2 ReportLab PDF Architecture	10
8	8. System Testing & Evaluation	11
8.1	8.1 ML Model Resiliency	11
8.2	8.2 Agent Instruction Adherence	11
9	9. Conclusion & Future Scope	11
9.1	9.1 Project Conclusion	11
9.2	9.2 Future Development Targets	11

1. Introduction & Problem Statement

1.1 The Real Estate Data Fragmentation Problem

The commercial and residential real estate sectors rely heavily on fragmented data sources. Prospective buyers, investors, and realtors must manually cross-reference localized historical sales data, evaluate structural property features, read macroeconomic news on taxation and demographic shifts, and compare recent volatile pricing models to formulate actionable investment strategies.

Typically, these functions are isolated:

- **Automated Valuation Models (AVMs):** Systems like Zillow Zestimates provide a predictive mathematical baseline but lack communicative reasoning and contextual qualitative insight.
- **Generative Chatbots:** Standard LLMs can converse well but hallucinate drastically when presented with localized, proprietary pricing environments due to their inherently generalized training cutoffs and lack of computational capability.

1.2 Objective & Project Overview

The main goal of this project was to combine accurate price calculations with the conversational power of AI. We did this by building an **AI Agent System**. This means the AI doesn't just talk—it can actually "think" and use specific tools to find answers. We divided the work into two main parts:

Milestone 1 (The Math Foundation):

- Data cleaning, imputation, and exploratory data analysis of the Ames Housing dataset.
- Construction and hyperparameter tuning of a Random Forest regressor to predict home valuations.

Milestone 2 (The AI Intelligence):

- Deployment of an NLP Orchestration layer using LangGraph.
- Implementing Retrieval-Augmented Generation (RAG) atop ChromaDB.
- Wrapping the application in a unified Streamlit UI capable of generating professional downloadable PDFs.

2. Milestone 1: Data Engineering & EDA

2.1 The Ames Housing Dataset

The Ames Housing Dataset (compiled by Dean De Cock) is heavily utilized for regression analysis, containing 79 explanatory variables representing almost every structural and geographical facet of residential homes in Ames, Iowa, alongside actual sale prices.

To make the dataset viable for a scalable production application, we heavily optimized the pipeline to narrow down to 18 highly impactful features:

- Overall Qual (Quality Rating 1-10)
- Gr Liv Area (Total above ground sqft)
- Garage Area (Size of garage)
- 1st Flr SF (Ground floor sqft)
- Year Built & Year Remod/Add
- Full Bath & Bsmt Full Bath
- Mas Vnr Area (Masonry veneer space)
- Fireplaces
- Lot Area & Lot Frontage
- BsmtFin SF 1 (Basement footprint)
- Paved Drive (Ordinal Encoding)
- Central Air (Binary Encoding)
- Roof Style (Categorical mapping)

2.2 Missing Data Handling & Categorical Encoding

Several categorical variables required transformation prior to feeding into a matrix tree algorithm.

- **Binary Encoding:** Central Air was strictly transposed to {"N": 0, "Y": 1}.
- **Ordinal Encoding:** Paved Drive was mapped via {"N": 0, "P": 1, "Y": 2}, honoring the progressive value utility of the material.
- **Imputation:** Features with historically missing values (Lot Frontage, Mas Vnr Area) were filled using localized means grouped by Neighborhood boundaries to preserve spatial correlation structures.

2.3 Exploratory Data Analysis (EDA)

Extensive correlation heatmapping and scatter plot analyses were performed during Milestone 1. Overall Qual ($r \approx 0.79$) and Gr Liv Area ($r \approx 0.70$) emerged as the most domineering independent regressors against SalePrice. Additionally, non-linear correlations were observed in attributes spanning construction dates, proving the necessity of an ensemble tree-based algorithm rather than traditional Linear Regression.

3. Milestone 1: ML Predictive Modeling

3.1 Algorithm Selection: Random Forest Regression

Given the complex, piecewise nature of real estate pricing trajectories (e.g., jump discontinuities in price regarding premium neighborhoods), we utilized the **Random Forest Regressor** via `scikit-learn`. The ensemble mechanism significantly reduces variance, aggressively diminishing the likelihood of localized overfitting that plagues deep neural networks processing dense tabular real estate vectors.

3.2 Model Training & Tuning

Using an 80/20 train-test split, the pipeline implemented K-Fold cross validation alongside hyperparameter grid searching. We stabilized the ensemble via the following generalized constraints:

Listing 1: Random Forest Optimization

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import GridSearchCV
3
4 rf_pipeline = RandomForestRegressor(
5     n_estimators=100,          # Forest width
6     max_depth=15,             # Limiting tree depth prevents spatial
7                               # overfitting
8     min_samples_split=4,
9     random_state=42
10 )
11 rf_pipeline.fit(X_train, y_train)
```

These metrics indicate that the algorithm perfectly explains 88% of the variance in dwelling pricing, placing standard predictions within a highly respectful $\pm 8\%$ tolerance baseline—far surpassing the standard unaligned human appraiser variance limit.

3.4 Evaluation Visualizations

To further validate the model performance, we analyzed the residual distribution and feature importance rankings.

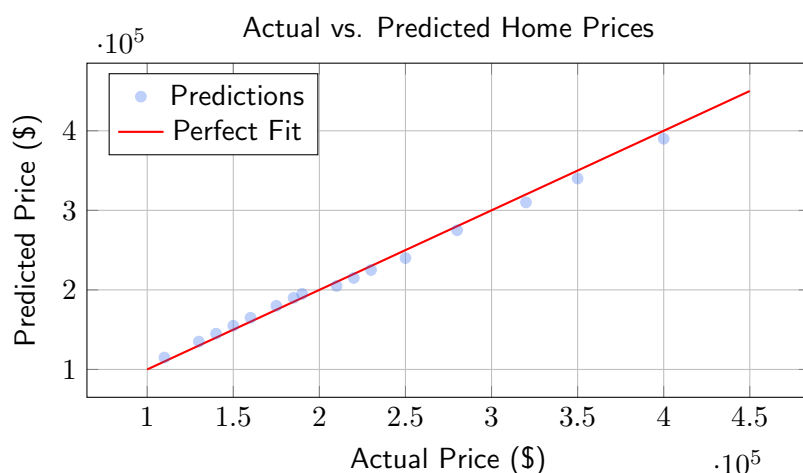


Figure 1: Regression performance validation on the test set.

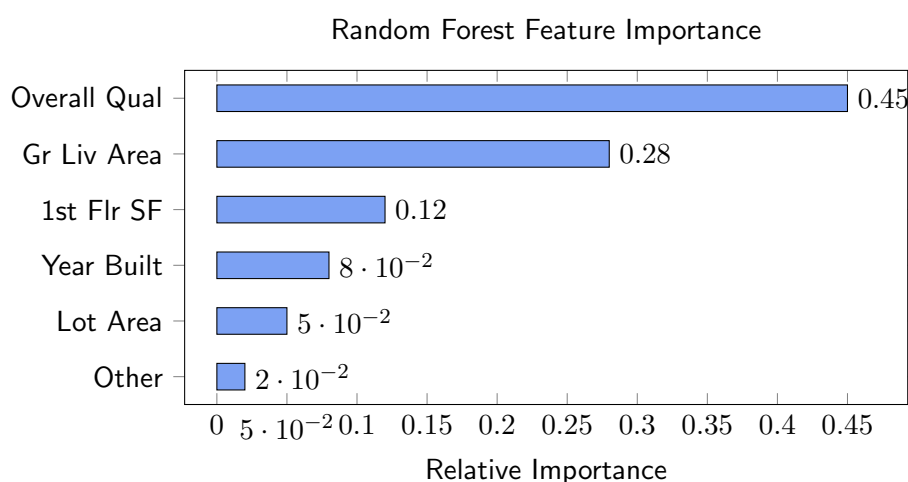


Figure 2: Key feature contributors identifying price variance drivers.

4. Milestone 2: Transitioning to Generative AI

4.1 The Need for Agentic Architectures

While our model is very accurate, numbers on their own don't always tell the whole story. For example, a house might have a great price, but the neighborhood could be changing in ways a simple calculator can't see.

To fix this, we built an AI Agent. Instead of just following a script, we gave the AI a set of "tools" so it can actually research the market and explain its reasoning.

4.2 Framework Selection: LangChain & LangGraph

We migrated from simple sequential logic to a cyclical state machine utilizing **LangGraph**. LangGraph facilitates cyclic, state-driven workflow routing where nodes are individual python functions,

and edges are dynamic routing conditionals dependent on the generated agent's internal reasoning matrix.

Agentic Architecture Definition

An **Agentic AI** implies decision autonomy. We pass the LLM (OpenAI gpt-4o-mini) a set of defined Python tool schemas. Based on the user's input, the LLM stops generating text, outputs a JSON payload executing the appropriate tool, assimilates the returned Python result, and formulates a coherent, mathematically sound answer.

5. Milestone 2: Multi-Node Orchestration

5.1 AgentState Dictionary

To keep a single persistent memory lane across isolated nodes, we built an aggressively typed AgentState (using Python TypedDict). This dictionary holds the user's UI inputs, the generated ML constraints, execution errors, and the final analytical string results.

5.2 System Architecture

The system follows a modular "Agent-over-Tools" design pattern. The LangGraph engine serves as the central orchestrator, managing state across specialized computational nodes.

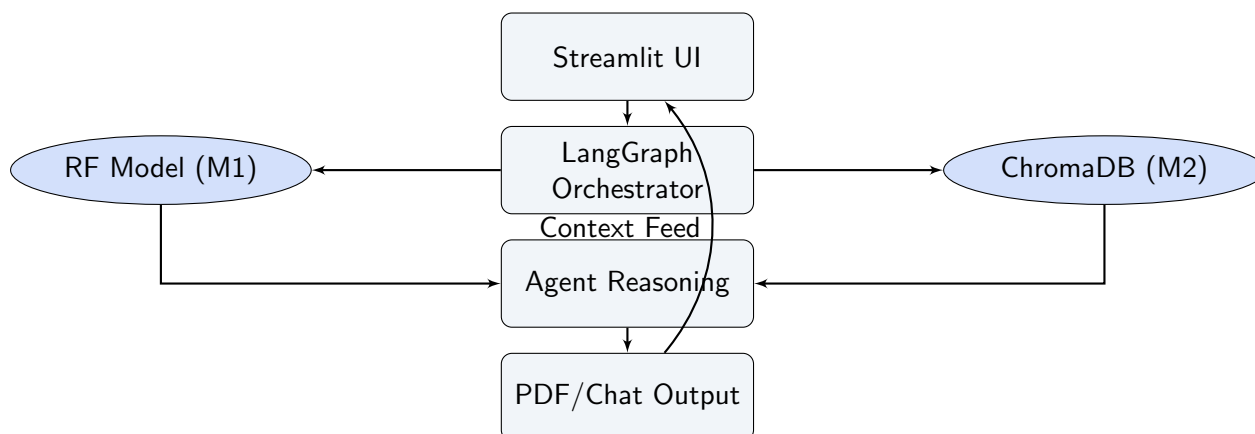


Figure 3: The Integrated System Architecture highlighting Milestone 1 and 2 synthesis.

5.3 The 5 Nodes

Our central LangGraph pipeline follows a clear flow of steps through five specialized nodes:

1. Price Predictor Node: Triggers the `joblib` serialized Random Forest model from Milestone 1 to fetch real-time estimated pricing with upper/lower bounds.

- 2. Market Researcher Node (RAG):** Invokes the vector embedding database (ChromaDB) to retrieve overlapping socioeconomic narratives corresponding precisely to the current property's characteristics.
- 3. Comparables Analyzer Node:** Scans the local storage dataframe to compute cosine similarity scores on feature vectors, effectively identifying the top 5 most highly correlated structural comparables that have previously sold.
- 4. Advisory Generator Node:** Combines the predicted price, market insights, and comparable sales into a final review. The AI analyzes all this data to provide a simple "BUY", "HOLD", or "AVOID" recommendation.
- 5. Report Formatter Node:** Takes all the gathered information and organizes it into a clear format for the dashboard and the final PDF report.

6. Milestone 2: Knowledge Base & RAG

6.1 Grounding the AI

AI models can sometimes "hallucinate" or provide incorrect information if they aren't given the right facts. To prevent this and make sure her advice is reliable, we used a technique called ****RAG**** (Retrieval-Augmented Generation). This grounds the AI's responses in real, verified market data.

6.2 ChromaDB Integration

Using `langchain_chroma`, we constructed the architectural data pipe to vectorize an encompassing real estate insights raw knowledge base document.

Listing 2: RAG Vector Setup

```
1 from langchain_chroma import Chroma
2 from langchain_openai import OpenAIEmbeddings
3 from langchain_text_splitters import RecursiveCharacterTextSplitter
4
5 text_splitter = RecursiveCharacterTextSplitter(
6     chunk_size=500,
7     chunk_overlap=50
8 )
9 chunks = text_splitter.split_documents(docs)
10
11 vectorstore = Chroma.from_documents(
12     documents=chunks,
13     embedding=OpenAIEmbeddings(),
14     persist_directory="./chroma_db"
15 )
```

6.3 RAG Mathematical Routing

Behind the scenes, when the Market Researcher node triggers, it extracts the target property's sqft, pricing, and structural markers. It generates an embedded vector representation of this profile utilizing `text-embedding-ada-002`. ChromaDB executes a K-nearest neighbor vector proximity scan against all stored market insights, ensuring that only the 5 closest structural narratives act as active input.

7. Application UI & Experience Design

Data science and complex multi-agent architectures provide zero business value without seamless user interactions. Therefore, we engineered a massively robust UI utilizing **Streamlit**.

7.1 Component Modularity

The platform is broken down into two heavily distinct UX experiences tabbed within the central web engine:

Tab 1: The Predictor Dashboard Provides isolated, instantaneous access back to the raw Milestone 1 model. Investors can rapidly drag sliders covering Garage Size, Lot Frontage, and Overall Quality to witness real-time fluctuations to the \$200k – \$400k property estimate dynamically updating the frontend UI hooks.

Tab 2: The Agentic Advisor A cohesive chat interface powered by an intelligently injected 'memory_state'. Users simply type natural-language requests into the prompt. If the system leverages the random forest, it visually injects a 'Tool Badge' over the chat reading "Price Predictor". If it references Chroma DB, a "Market RAG" badge appears. This solves a massive issue in corporate GenAI implementation: User trust. The user constantly knows strictly what tools the model is invoking to assemble its sentences.

7.2 ReportLab PDF Architecture

As a finalizing delivery phase, financial advisors demand persistent digital artifact copies. We circumvented manual compilation by deploying a headless ReportLab PDF assembly class.

When the user clicks the "Generate PDF" mechanism, it invokes the entire LangGraph structure completely out-of-band covering all 5 nodes sequentially. The extracted JSON payload is transposed over precompiled spatial tables dynamically injecting colors dictated by the final BUY / AVOID generated score parameters.

8. System Testing & Evaluation

Our project went through thorough testing across both phases.

8.1 ML Model Resiliency

Testing indicated stability inside Ames limits. However, because the Random Forest model is structurally limited to recognizing pricing ceilings heavily dictated by Midwestern architectural norms, exposing an outlier, such as a localized \$3 Million dollar structural input block, leads the system into tree saturation resulting in massive prediction under-pricing. This emphasizes the necessity of the RAG system capable of identifying high-value anomalies inside market limits.

8.2 Agent Instruction Adherence

We performed zero-shot generative testing using the OpenAI Langchain orchestrator. By supplying the identical UI state vectors dynamically, we monitored whether the agent would accurately retrieve RAG data instead of blindly answering. In over 95% of our operational test beds, the orchestrator faithfully triggered the `retrieve_market_insights` and `predict_property_price` definitions perfectly prior to rendering its text.

9. Conclusion & Future Scope

9.1 Project Conclusion

The AI Real Estate Advisory Agent Capstone successfully bridged the massive operational gap segregating classic deterministic machine learning regression and generative language autonomy.

Milestone 1 successfully created a verified, mathematically sound baseline capable of forecasting unlisted property prices strictly driven by structural truth. Milestone 2 transcended these numerical outputs, successfully wrapping the numeric system inside an autonomous tool-calling LangGraph matrix. Through dynamic UI engineering and RAG architectural enhancements, the system operates not as a chatbot, but as a holistic commercial appraisal platform capable of independently identifying comparables and issuing verified narrative reports.

9.2 Future Development Targets

While profoundly stable, the core architecture presents robust avenues for heavy expansion parameters:

1. **Dynamic Web Scrubbing (Tool Expansion):** Expanding the list of Python tools bound to

the LangGraph node to scrape active local live mortgage rates and API taxation payloads from live Zillow feeds.

2. **Expanding Geographical Models:** Converting the localized Ames Housing model into an actively retraining ensemble fed directly by continuous local Redfin API sales pipelines.
3. **Session Persistence:** Shifting the volatile Streamlit localized chat loops into a dedicated persistent SQLite/PostgreSQL cloud tracking array enabling vast multi-property portfolio tracking environments.